

Great topic for your class! Here are **concise Python code snippets** that demonstrate each list operation you mentioned, with **what it does**, **time complexity**, and a **mini example** your students can run immediately.

Tip for teaching: Have students print the list **before and after** each operation to see the effect.

1) Insertion at End — .append(x)

Description: Adds x to the end

Complexity: O(1) amortized

Use case: Building a buffer of sensor readings

```
1 # append: O(1) amortized
2 data = [10, 20, 30]
3 print("Before:", data)
4
5 data.append(42) # add one item to the end
6 print("After:", data) # [10, 20, 30, 42]
```

2) Insertion at Index — .insert(i, x)

Description: Inserts x at index i (shifts others to the right)

Complexity: O(n) (items after i must shift)

Use case: Rare; avoid in large loops

```
1 # insert: O(n)
2 data = ["second", "third"]
3 print("Before:", data)
4
5 data.insert(0, "first") # insert at the beginning
6 print("After:", data) # ["first", "second", "third"]
```

3) Extend — .extend(iterable)

Description: Appends all items from the iterable

Complexity: O(k) where k = len(iterable)

Use case: Merging batch data packets

```
1 # extend: O(k)
2 data = [1, 2]
3 print("Before:", data)
```

```
4
5 data.extend([3, 4, 5])    # add multiple elements
6 print("After:", data)    # [1, 2, 3, 4, 5]
```

Contrast with `append([3,4,5])` which would add a **single list** as one element.

4) Removal (Last) — `.pop()`

Description: Removes & returns the last item

Complexity: $O(1)$

Use case: Stack (LIFO) behavior

```
1 # pop last: O(1)
2 data = [100, 200, 300]
3 print("Before:", data)
4
5 last = data.pop()    # removes 300
6 print("Popped:", last)
7 print("After:", data) # [100, 200]
```

5) Removal (Index) — `.pop(i)`

Description: Removes & returns the item at index `i`

Complexity: $O(n)$ (items after `i` shift left)

Use case: Special cases only

```
1 # pop at index: O(n)
2 data = ["a", "b", "c", "d"]
3 print("Before:", data)
4
5 first = data.pop(0)    # removes from the front (slow for big
6 print("Popped:", first)
7 print("After:", data)  # ["b", "c", "d"]
```

6) Removal by Value — `.remove(x)`

Description: Removes **first occurrence** of `x`

Complexity: $O(n)$ (search + shift)

Use case: Filter out a known value

```
1 # remove by value: O(n)
```

```
2 data = [2, 3, 3, 4]
3 print("Before:", data)
4
5 data.remove(3)      # removes only the first 3
6 print("After:", data) # [2, 3, 4]
```

If x isn't present, ValueError is raised.

7) Search — .index(x)

Description: Returns index of first occurrence of x

Complexity: O(n)

Use case: Find position of a delimiter/marker

```
1 # index: O(n)
2 data = ["start", "middle", "end"]
3 print("Data:", data)
4
5 pos = data.index("middle") # 1
6 print("Position of 'middle':", pos)
```

Use if x in data: before .index(x) to avoid ValueError.

8) Sorting — .sort()

Description: In-place sort (Timsort)

Complexity: O(n log n)

Use case: Organize timestamps, scores, etc.

```
1 # sort: O(n log n), in-place
2 data = [5, 2, 8, 1]
3 print("Before:", data)
4
5 data.sort()          # ascending
6 print("Ascending:", data)
7
8 data.sort(reverse=True) # descending
9 print("Descending:", data)
```

Use sorted(data) to create a **new sorted list** without changing the original.

9) Copy — .copy()

Description: Shallow copy

Complexity: $O(n)$

Use case: Preserve original for comparison

```
1 # copy (shallow):  $O(n)$ 
2 original = [[1, 2], [3, 4]]
3 backup = original.copy()
4
5 backup[0].append(99) # modifies inner list (shared!)
6 print("Original:", original) # [[1, 2, 99], [3, 4]]
7 print("Backup:", backup)     # [[1, 2, 99], [3, 4]]
```

For nested structures, use `copy.deepcopy` to avoid shared inner objects.

10) Clear — .clear()

Description: Removes all items

Complexity: $O(n)$

Use case: Reset a buffer

```
1 # clear:  $O(n)$ 
2 data = [1, 2, 3]
3 print("Before:", data)
4
5 data.clear()
6 print("After:", data) # []
```

Bonus: Append vs Extend vs Insert (Quick Contrast)

```
1 a = [1, 2]
2
3 a.append([3, 4]) # -> [1, 2, [3, 4]]
4 print("append:", a)
5
6 b = [1, 2]
7 b.extend([3, 4]) # -> [1, 2, 3, 4]
8 print("extend:", b)
9
10 c = [1, 2]
11 c.insert(1, 99) # -> [1, 99, 2]
```

```
12 print("insert:", c)
```

Mini Exercises (with expected outputs)

1. **Build a stack** using `append` and `pop()` and print items as you pop.
2. Given `data = [5, 3, 5, 2, 5]`, remove **only the first** 5. What remains?
3. Merge two lists `a=[1,2]` and `b=[3,4,5]` into a **without** nesting.
4. Sort `scores = [50, 90, 70, 60]` descending, **without** modifying the original.
5. Make a shallow copy of `matrix = [[0], [1]]` and show how modifying `copy[0]` affects `matrix`.

Answer hints (brief):

1. `append` → `pop` prints in reverse order (LIFO).
 2. `[3, 5, 2, 5]`.
 3. `a.extend(b)`.
 4. `sorted(scores, reverse=True)` returns a new list.
 5. Shallow copy shares inner lists; both reflect the change.
-

If you'd like, I can bundle these into a **printable handout (PDF/Word)** with a one-page **cheat sheet** and **practice problems with solutions**—perfect for your IVORY cohort or your Python class.